

Assignment 1: Business Understanding, Event Design & Data Generation

Onboarding Flow Optimization A/B Test

Aishwarya Mishra

August 2026

Table of Contents

Abstract

1. Introduction and Business Context

- 1.1 Product and User Journey
- 1.2 Problem Statement
- 1.3 Opportunity Sizing

2. Hypothesis

3. Experimental Design

- 3.1 Design Summary
- 3.2 Arms
- 3.3 Step Classification (Confound Control)
- 3.4 Sample Size and Power (Design-Stage Reasoning)
- 3.5 Threats to Validity and Mitigations

4. KPI Framework

- 4.1 Primary Metric
- 4.2 Secondary (Supporting) Metrics
- 4.3 Guardrail Metrics
- 4.4 Metric Governance

5. Assumptions and Limitations

6. Data Platform Requirement

Abstract

1. Design Principles

2. Envelope Specification

3. Identity, Randomization, and Assignment Integrity

4. Data Governance and Privacy Posture

5. Event Catalog

- 5.1 Funnel and Onboarding Events
- 5.2 Product Engagement Events
- 5.3 Monetization Events
- 5.4 Quality and Guardrail Events

6. Per-Event Validation Rules

7. Schema Versioning and Evolution

8. Metric-to-Event Lineage

Abstract

The organization under study operates a B2C mobile application that teaches English using AI-driven instruction. New users are routed through a thirteen-step onboarding sequence before reaching the core product surface. Observed completion of this sequence (60%) trails the stated industry benchmark (approximately 80%) by twenty percentage points, while the downstream paid-conversion rate of users who *do* complete onboarding (6%) is in line with industry norms. This document establishes the business case, the causal hypothesis under test, the experimental design used to test it, and the KPI/guardrail framework against which the experiment's result will be judged. It is the analytical foundation for the data platform built in Assignments 2 through 4; every metric defined here maps onto a concrete column or aggregation in the warehouse and data mart layers described in Assignment 2.

1. Introduction and Business Context

1.1 Product and User Journey

The product is a mobile, subscription-based English-language learning application. A new user's journey has three phases: (i) **acquisition** — the user installs the app, typically via paid or referral channels; (ii) **onboarding** — a sequence of account-creation, assessment, and preference-capture steps that precede any exposure to the core product; and (iii) **engagement and monetization** — lesson consumption, retention, and eventual conversion to a paid subscription. This assignment, and the platform it specifies, is concerned with phase (ii) and its causal effect on phase (iii).

1.2 Problem Statement

Every additional step interposed between install and first product value is an additional point of attrition. Because acquisition is typically a paid activity (advertising spend, referral incentives), attrition inside onboarding is economically distinct from attrition earlier in the funnel: it represents a loss of *already-paid-for* users, not a failure to acquire. The organization's stated position is that its current thirteen-step flow is the primary driver of the observed 20-percentage-point completion gap relative to industry benchmarks, and that the checkout/monetization mechanics downstream of onboarding are *not* the problem, since paid conversion of the users who do complete onboarding already matches industry norms.

1.3 Opportunity Sizing

Using the organization's illustrative planning assumptions — 100,000 new users entering onboarding per month, and \$30 average monthly revenue per paying user — the revenue impact of closing the completion gap can be expressed as a simple identity:

$$\text{Monthly Revenue} = \text{Users_Started} \times \text{Completion_Rate} \times \text{Paid_Conversion_Rate} \times \text{ARPU}$$

Scenario	Completion Rate	Users Completing	Paying Users (@6%)	Monthly Revenue
Current state	60%	60,000	3,600	\$108,000

Scenario	Completion Rate	Users Completing	Paying Users (@6%)	Monthly Revenue
Industry-benchmark state	80%	80,000	4,800	\$144,000
Delta	+20 pp	+20,000	+1,200	+\$36,000/mo (~\$432,000/yr)

This identity makes explicit the assumption the experiment is designed to test: that `Completion_Rate` can be moved by intervention on onboarding step count *without* a compensating decrease in `Paid_Conversion_Rate` or ARPU — i.e., that the funnel stages are not causally entangled through some confound such as user quality or personalization depth. Section 5 (Guardrail Metrics) exists specifically to detect a violation of that assumption.

2. Hypothesis

Primary hypothesis (H1): Reducing the number of sequential onboarding steps from thirteen will causally reduce onboarding attrition, increasing the onboarding completion rate, and this effect will propagate (at least partially) into activation and paid-conversion volume, because a fixed downstream conversion rate applied to a larger completing population yields more paying users in absolute terms.

Identification concern: A naive test of “shorter onboarding is better” conflates two distinct manipulations — *how many* steps a user must pass through, and *which* steps are removed. If the removed steps happen to be ones that meaningfully improve personalization or set correct user expectations, an observed lift in completion could co-occur with a decline in downstream quality, and the experiment would misattribute a compositional effect to a pure friction effect. The design in Section 3 addresses this by deliberately holding a “must-have” step set constant across all three variants and manipulating only the “can-wait” and “nice-to-have” tiers (see Table 3.1), so any observed guardrail movement can be attributed to *specific, named* removed steps rather than to step count in the abstract.

Null hypothesis (H0): Onboarding step count has no causal effect on completion rate within the range tested (5–13 steps); observed differences between arms are attributable to sampling variation.

3. Experimental Design

3.1 Design Summary

Parameter	Value
Design type	Three-arm, between-subjects, randomized controlled trial
Randomization unit	Individual user, at first app open
Assignment mechanism	Uniform random draw, ~33.3% per arm
Stratification	Platform (iOS / Android), balanced within each arm
Primary endpoint	Onboarding completion rate
Secondary endpoints	Activation rate, D1/D7 return, paid conversion rate, time-to-first-lesson

Parameter	Value
Guardrail endpoints	Lesson relevance score, D30 return, refund rate, support-ticket rate, notification opt-in rate
Platform coverage	iOS and Android, balanced

3.2 Arms

Arm	Label	Step Count	Design Intent
A	Control	13 (unchanged)	Establishes the observational baseline; no product change
B	Medium	7	Tests a moderate reduction, retaining most “can-wait” steps
C	Short	5	Tests an aggressive reduction, retaining only “must-have” steps

3.3 Step Classification (Confound Control)

Each of the thirteen steps in the master flow is classified into exactly one of three buckets, and this classification — not an ad hoc truncation of the step list — determines what each arm removes:

Bucket	Definition	Treatment across arms
Must-have	Structurally required before an account exists or a lesson can be served (e.g., account creation, level assessment)	Present in A, B, and C
Can-wait	Useful, but deferrable to after the user has experienced product value	Present in A and (partially) B; deferred or omitted in C
Nice-to-have	Marginal value; combinable into other surfaces or removable with limited loss	Present only in A

This scheme means B and C are not simply “the first k steps of A” — they are principled subsets, so a completion-rate difference between A and C can be decomposed, at least partially, into “steps removed because they were nice-to-have” versus “steps removed because they were merely deferred,” supporting a more defensible causal claim than step-count alone would allow. The full step-to-bucket-to-arm mapping is version-controlled as data ([stepmap.json](#)) rather than hard-coded in analysis scripts, so it can be audited and reused identically by the data generator (Assignment 1), the warehouse (Assignment 2), and the dashboard (Assignment 4).

3.4 Sample Size and Power (Design-Stage Reasoning)

Although this assignment uses simulated rather than production traffic, the design is specified as if for production: with a baseline completion rate of 60% and a targeted minimum detectable effect (MDE) of 5 percentage points at $\alpha = 0.05$ (two-sided) and 80% power, a two-proportion z-test requires approximately

1,450-1,500 users per arm. At the illustrative volume of 100,000 new users/month split three ways, this sample size is reached within hours of launch, meaning the primary metric is not statistically under-powered; the binding constraint on run duration is instead the guardrail metrics with lower base rates (e.g., refund rate, support-ticket rate), which require materially larger samples and a longer observation window (the D30 guardrail alone requires at least a 30-day observation tail per cohort). Assignment 4 will apply this reasoning against the actual generated sample.

3.5 Threats to Validity and Mitigations

Threat	Description	Mitigation in this design
Sample Ratio Mismatch (SRM)	Unequal realized allocation across arms indicates a broken randomizer or differential instrumentation loss	<code>mart_kpi_by_group</code> reports raw <code>users_started</code> per arm for an SRM check before any effect is interpreted (Assignment 4)
Novelty effect	Short-term behavioral change due to novelty of a new flow, not a durable effect	D30 return is tracked specifically to distinguish durable retention from a novelty spike
Instrumentation asymmetry	If variant-specific code paths log events differently, metrics become incomparable	All three arms share one event schema (Assignment 1, Section on Event Catalog); no variant-specific event types exist
Interference / network effects	Not applicable at this scale; users of a language-learning app do not materially interact with one another inside the product	Not mitigated further; assumed negligible
Multiple comparisons	Testing one primary + four secondary + five guardrail metrics inflates false-positive risk across the full metric set	Primary metric is pre-registered as the sole basis for the ship/no-ship decision; guardrails are interpreted as a directional risk check, not independently hypothesis-tested, in Assignment 4

4. KPI Framework

4.1 Primary Metric

Onboarding completion rate — the proportion of users who start onboarding and reach `onboarding_completed`.

```
Completion Rate = COUNT(DISTINCT user_id WHERE onboarding_completed)
                  / COUNT(DISTINCT user_id WHERE onboarding_started)
```

This is the sole metric on which the ship/no-ship decision is made. It is chosen as primary — rather than, say, paid conversion — because it is the metric closest to the mechanism under test (step count) and least confounded by downstream factors (pricing, payment friction) outside this experiment's scope.

4.2 Secondary (Supporting) Metrics

Metric	Formal Definition	Business Purpose
Activation rate	<code>first_lesson_completed</code> users divided by <code>onboarding_completed</code> users	Confirms completers are engaging with the product, not merely clicking through a shorter flow without absorbing it
Day-1 / Day-7 return rate	Users with <code>app_opened_dayN</code> divided by <code>onboarding_completed</code> users	Early behavioral signal of retained interest
Paid conversion rate	<code>subscription_purchased</code> users divided by <code>first_lesson_completed</code> users	Connects the funnel to revenue; expected to remain flat across arms under H1
Time to first lesson	<code>first_lesson_started.timestamp</code> - <code>onboarding_started.timestamp</code>	Direct, continuous measure of friction removed; complements the binary completion metric

4.3 Guardrail Metrics

Guardrails exist to detect the identification concern in Section 2: that a completion-rate lift purchased by removing personalization-relevant steps could be a false economy.

Guardrail	Formal Definition	Failure Mode It Detects
Lesson relevance score	Mean of self-reported <code>relevance_score</code> (1-5) on <code>lesson_feedback_submitted</code>	Fewer personalization questions leads to generic content and user dissatisfaction not visible in completion rate
Day-30 return rate	Users with <code>app_opened_day30</code> divided by <code>onboarding_completed</code> users	Shorter flow could admit low-intent users who complete easily but churn once novelty fades
Refund rate	<code>refund_requested</code> users divided by <code>subscription_purchased</code> users	Rushed onboarding leads to a user misunderstanding what was purchased
Onboarding-related support-ticket rate	Users with <code>support_ticket_created</code> (onboarding-tagged) divided by <code>onboarding_started</code> users	A skipped explanatory step (billing, permissions) manifests as downstream confusion
Notification opt-in rate	Users with <code>notification_permission_accepted</code> divided by users with <code>notification_permission_shown</code>	If the permission step is deferred out of the primary flow, opt-in typically falls, weakening future re-engagement capacity

4.4 Metric Governance

No guardrail is a hard stop encoded in the platform itself; each is surfaced to the analyst in the Assignment 4 dashboard alongside the primary metric, with the interpretive judgment (ship, ship-with-mitigation, or hold) left to human decision-makers. This is a deliberate design choice: automatically gating a launch decision on

five simultaneously-measured guardrails, each with its own base-rate noise, would itself introduce a multiple-comparisons problem the design in Section 3.5 explicitly avoids compounding.

5. Assumptions and Limitations

1. **Simulated, not production, data.** The dataset analyzed in Assignments 2-4 is synthetically generated (see [data_generator.py](#)) to match the funnel shape and guardrail directions implied by this document, since production instrumentation does not yet exist. Effect sizes in the simulated data are illustrative, calibrated to be directionally consistent with the business hypothesis, not empirically estimated from real users.
2. **Flat paid-conversion assumption.** The opportunity-sizing model in Section 1.3 assumes [Paid_Conversion_Rate](#) is invariant to onboarding step count. This is the assumption under test, not a given; if guardrail analysis in Assignment 4 shows it does not hold, the revenue estimate in Section 1.3 is an upper bound, not a forecast.
3. **Single-app, single-market scope.** No geographic, language-cohort, or platform-version stratification beyond iOS/Android is modeled. Effects may not generalize across markets with different device mixes or payment norms.
4. **No network or peer effects modeled,** consistent with the product's single-player usage pattern.
5. **Guardrails are directional signals, not hypothesis tests,** per Section 4.4; this document does not claim statistical significance for guardrail movements, only that the platform is instrumented to detect them.

6. Data Platform Requirement

Testing H1 while respecting the guardrails in Section 4.3 requires a pipeline capable of: (i) capturing every funnel and product event at user-level granularity, tagged with arm, platform, and timestamp; (ii) validating and landing that data reliably even in the presence of malformed or duplicate delivery; (iii) modeling it into a warehouse that supports both the primary metric and every guardrail without ad hoc re-querying of raw events; and (iv) serving it to a dashboard and a statistical test suite. These four requirements map directly onto Assignments 1-4 respectively; Assignment 2 (immediately following) implements (ii) and (iii).

See [02_event_tracking_plan.md](#) for the event schema and governance model, and [data_generator.py](#) for the synthetic dataset used to develop and validate the pipeline ahead of real production traffic.

Abstract

This document specifies the event-level instrumentation required to operationalize the KPI framework defined in [01_business_understanding.md](#). It defines a schema, a naming taxonomy, per-event validation rules, randomization and identity semantics, a versioning strategy, and the explicit lineage from raw events to each KPI and guardrail. The schema is implemented in machine-readable form as [event_schema.json](#) and [stepmap.json](#), which are consumed directly — not merely referenced — by the data generator, the warehouse build in Assignment 2, and (prospectively) the dashboard in Assignment 4, so this document and the running system cannot silently drift apart.

1. Design Principles

- 1. Long (narrow) format.** One row per event, not one row per user with dozens of event-specific columns. This trades some query-time convenience (addressed by the warehouse layer in Assignment 2) for schema flexibility: a new event type is a new value in `event_name`, not a schema migration.
- 2. Uniform envelope, variable payload.** Every event shares seven identical top-level fields (Section 2) regardless of type; type-specific detail is isolated in a `properties` object. This keeps the raw table narrow and lets generic tooling (deduplication, PII scanning, routing) operate without per-event-type branching.
- 3. Deterministic naming taxonomy.** Every event name follows `object_verb_pasttense` in `snake_case` (e.g., `onboarding_step_completed`, not `CompleteStep` or `step-complete`). Past tense signals that the event is an immutable fact about something that already happened, not a command or a mutable state.
- 4. Idempotent identity.** Every event carries a client-generated `event_id` (UUIDv4) at creation time, independent of when or how many times it is delivered to the backend. This is what allows the staging layer (Assignment 2) to de-duplicate retried deliveries safely.
- 5. No event without an owning user and timestamp.** `user_id` and `event_timestamp` are non-nullable by contract; violations are routed to a quarantine table rather than silently dropped or silently accepted (Assignment 2, Section on Data Quality).

2. Envelope Specification

Field	Type	Nullable	Description
<code>event_id</code>	string (UUIDv4)	No	Client-generated unique identifier; the basis for deduplication
<code>user_id</code>	string	No	Pseudonymous identifier, stable for the user's lifetime in the dataset
<code>event_name</code>	string (enum)	No	One of the catalog names in Section 5
<code>event_timestamp</code>	string (ISO-8601, UTC)	No	Client-side event time, not server ingestion time
<code>test_group</code>	string (enum: <code>A</code> , <code>B</code> , <code>C</code>)	No	Assigned onboarding arm; set once per user at randomization and copied onto every subsequent event for that user
<code>platform</code>	string (enum: <code>iOS</code> , <code>Android</code>)	No	Device platform
<code>session_id</code>	string	Yes	Groups events emitted within one continuous app session
<code>properties</code>	object (JSON)	Yes	

Field	Type	Nullable	Description
			Event-specific attributes; schema per <code>event_name</code> given in Section 5

`test_group` and `platform` are denormalized onto every event, not only onto a separate assignment record, deliberately: it allows any single event-level query (e.g., a step-abandonment funnel) to be sliced by arm without a join back to a user dimension, at the cost of redundant storage — a standard trade-off for high-cardinality, append-only event streams.

3. Identity, Randomization, and Assignment Integrity

- `user_id` is minted at `app_installed` and is treated as immutable for the remainder of the user's event history in this dataset. No identity-merge or cross-device-stitching logic is in scope.
- Arm assignment happens exactly once, at `onboarding_started` (functionally the user's first app open), via a uniform random draw across `{A, B, C}`, stratified so that iOS and Android are balanced within each arm. Once assigned, `test_group` does not change for that user, including on reinstall (reinstall handling is out of scope for this dataset; each simulated user installs exactly once).
- **Sample Ratio Mismatch (SRM) as a pre-condition, not an afterthought.** Because every event carries `test_group`, an SRM check — `COUNT(DISTINCT user_id)` per arm compared against the expected 33/33/33 split — is a single `GROUP BY` against `mart_kpi_by_group` (Assignment 2) and is treated as a precondition for interpreting *any* other metric in Assignment 4: a broken randomizer invalidates every downstream comparison regardless of how favorable the primary metric looks.

4. Data Governance and Privacy Posture

- `user_id` is pseudonymous by construction (a generated identifier, not an email address, device advertising ID, or other directly identifying value), consistent with data-minimization practice for experimentation telemetry.
- No event in the catalog (Section 5) captures free-text user input beyond `refund_reason` and `exit_reason`, both of which are constrained to a fixed enumeration rather than free text, to avoid inadvertently collecting PII inside an analytics payload.
- `support_ticket_created` intentionally captures only a boolean relevance flag and a coarse `category` enum, not ticket contents; ticket contents live in the support system of record, not the analytics event stream.
- Retention policy (for a production deployment, out of scope for this academic dataset): raw event-level data would typically be retained for the duration of the experiment plus one full guardrail observation window (30 days post-experiment-close per Section 3.4 of the business- understanding document), after which only the aggregated data-mart tables would be retained.

5. Event Catalog

Each entry below states the triggering condition and its `properties` schema. The complete machine-readable form (used directly by `data_generator.py`) is `event_schema.json`.

5.1 Funnel and Onboarding Events

Event	Triggering Condition	properties
<code>app_installed</code>	App installed on device	<code>device_platform</code>
<code>onboarding_started</code>	User begins onboarding (arm assignment occurs here)	<code>total_steps_in_flow</code>
<code>onboarding_step_viewed</code>	Step rendered to the user	<code>step_number</code> , <code>step_name</code> , <code>step_bucket</code>
<code>onboarding_step_completed</code>	User successfully submits/advances past a step	<code>step_number</code> , <code>step_name</code> , <code>time_on_step_seconds</code>
<code>onboarding_step_abandoned</code>	User exits the app or session mid-step	<code>step_number</code> , <code>step_name</code> , <code>exit_reason</code> (enum: <code>closed_app</code> , <code>backgrounded</code> , <code>timeout</code> , <code>unknown</code>)
<code>onboarding_completed</code>	User reaches the end of their arm's step sequence	<code>total_time_seconds</code> , <code>steps_completed</code>
<code>account_created</code>	Account/profile persisted	<code>signup_method</code> (enum: <code>email</code> , <code>google</code> , <code>apple</code>)

5.2 Product Engagement Events

Event	Triggering Condition	properties
<code>first_lesson_started</code>	User begins their first lesson (activation trigger)	<code>lesson_id</code>
<code>first_lesson_completed</code>	User finishes their first lesson (activation success event)	<code>lesson_id</code> , <code>score</code>
<code>app_opened_day1</code> / <code>app_opened_day7</code> / <code>app_opened_day30</code>	Return visit at the corresponding horizon	<code>days_since_install</code>
<code>notification_permission_shown</code>	OS-level permission prompt displayed	—
<code>notification_permission_accepted</code> / <code>notification_permission_denied</code>	User responds to the prompt	—

5.3 Monetization Events

Event	Triggering Condition	properties
<code>trial_started</code>	Free trial begins	<code>trial_length_days</code>
<code>subscription_purchased</code>	User converts to a paying subscriber	<code>plan</code> (enum: <code>monthly</code> , <code>annual</code>), <code>price_usd</code>
<code>subscription_cancelled</code>	User cancels an active subscription	<code>days_since_purchase</code>
<code>refund_requested</code>	User requests a refund	<code>days_since_purchase</code> , <code>reason</code> (enum: <code>not_as_expected</code> , <code>too_expensive</code> , <code>found_free_alternative</code> , <code>other</code>)

5.4 Quality and Guardrail Events

Event	Triggering Condition	properties
<code>app_crash</code> / <code>app_error</code>	Technical fault during a session	<code>error_code</code> , <code>screen</code>
<code>support_ticket_created</code>	Support ticket logged (may originate in support tooling, not the app)	<code>is_onboarding_related</code> (boolean), <code>category</code> (enum)
<code>lesson_feedback_submitted</code>	User rates lesson relevance	<code>relevance_score</code> (integer, 1-5)

6. Per-Event Validation Rules

Applied at the staging layer (Assignment 2, `schema/02_staging.sql`); listed here as part of the schema contract rather than buried in SQL comments:

Rule	Applies To	Rejection Behavior
<code>event_id</code> , <code>user_id</code> , <code>event_name</code> , <code>event_timestamp</code> non-null	All events	Routed to <code>stg_events_rejects</code> , not dropped
<code>test_group</code> in {A, B, C}	All events	Routed to <code>stg_events_rejects</code>
<code>platform</code> in {iOS, Android}	All events	Routed to <code>stg_events_rejects</code>
<code>event_id</code> uniqueness	All events	First-seen row wins; subsequent duplicates dropped at staging
<code>step_number</code> present and in {1..13}	<code>onboarding_step_*</code> events	Enforced by construction in the generator; would be a staging check against a production source
<code>relevance_score</code> in {1..5}	<code>lesson_feedback_submitted</code>	Enforced by construction; production staging would range-check

7. Schema Versioning and Evolution

The schema is expressed as data (`event_schema.json`), not as inline logic, specifically so it can be versioned independently of the code that produces or consumes it. The evolution policy assumed for this project:

- **Additive changes** (a new event name, a new optional `properties` key) are non-breaking and require no coordination beyond updating `event_schema.json`.
- **Breaking changes** (renaming an event, changing a field's type, making an optional field required) would require a new `event_name` or a `schema_version` envelope field in a production system; this dataset does not exercise that path, since it is generated once against a single fixed schema version.
- The step-to-bucket-to-arm mapping (`stepmap.json`) is versioned separately from the event schema itself, because it is experiment-design metadata (which steps exist, and which arm sees them) rather than wire-format metadata (what an event looks like on the wire) — conflating the two would make it harder to run a second, differently-designed experiment against the same event schema in the future.

8. Metric-to-Event Lineage

Restated here from the business-understanding document, in the direction that matters for instrumentation review: for each KPI, the minimal set of events that must be reliably captured for the metric to be computable.

KPI	Required Events
Onboarding completion rate	<code>onboarding_started</code> , <code>onboarding_completed</code>
Activation rate	<code>onboarding_completed</code> , <code>first_lesson_completed</code>
Day-1 / Day-7 / Day-30 return rate	<code>onboarding_completed</code> (denominator), <code>app_opened_day1 / day7 / day30</code>
Paid conversion rate	<code>first_lesson_completed</code> , <code>subscription_purchased</code>
Time to first lesson	<code>onboarding_started</code> , <code>first_lesson_started</code>
Lesson relevance (guardrail)	<code>lesson_feedback_submitted</code>
Refund / cancellation rate (guardrail)	<code>subscription_purchased</code> , <code>refund_requested</code> , <code>subscription_cancelled</code>
Notification opt-in rate (guardrail)	<code>notification_permission_shown</code> , <code>notification_permission_accepted</code>
Onboarding support-ticket rate (guardrail)	<code>onboarding_started</code> , <code>support_ticket_created</code>

Any gap in this table — an event type dropped from instrumentation — is, by construction, a gap in the organization's ability to answer the business question in Section 1 of the business-understanding document. This table is therefore the acceptance criterion for “is the platform instrumented correctly,” independent of whatever the experiment's eventual result turns out to be.

See `event_schema.json` for the machine-readable schema, `stepmap.json` for the per-arm step definitions, and `data_generator.py` for the synthetic dataset that implements this specification end to end.